BUSINESS REQUIREMENTS DOCUMENT

WORKING DOCUMENT

Policy Quote Acceptance Prediction Solution
Data & Analytics Department

Caribe Insurance Group

Revision History:

Date	Version	Name	Description of Changes
Mar. 1, 2009	1.0	IAG Consulting	Initial Creation
Jan. 13 2026	2.0	Caribe Insurance Group	Adapted template for academic use; added project-specific content, business use case, and requirements.

TABLE OF CONTENTS

UNIT 1: INTRODUCTION

Executive Summary

Caribe Insurance Group is a fictional insurance company based on real industry practices. The company uses an internal software system to generate insurance quotes when customers request them. While the current system produces accurate quotes, it does not help Insurance Agents understand which customers are more likely to accept or decline a quoted policy.

To address this limitation, Caribe Insurance Group plans to develop the Quote Acceptance Prediction System (QAPS). This system will use machine learning to estimate the likelihood that a customer will accept or decline an insurance quote. The prediction results will be visible only to Insurance Agents and internal users, not to customers. This will help agents prioritize follow-ups and improve operational efficiency.

This Business Requirements Document (BRD) represents the first phase of the Systems Development Life Cycle (SDLC). Creating formal requirements early helps ensure that business needs are clearly defined before development begins, which is especially important for machine learning systems that depend on data and ongoing evaluation (Amershi et al., 2019).

Purpose of this Document

The purpose of this document is to describe the business needs and objectives for the Quote Acceptance Prediction System (QAPS). This BRD explains why the project is needed, what it aims to achieve, and what challenges may affect its success. The QAPS is intended to help Insurance Agents understand which customers are more likely to accept or decline an insurance quote.

The information provided by the system will support agents in deciding how quickly and how often to follow up with customers. The intended audience for this document includes business leaders, Insurance Agents, Systems Analysts, and the machine learning development team. Defining requirements early helps align stakeholders, clarify scope, and prevent misunderstandings (Reqtest, n.d.; EBSCO, n.d.).

Business Objectives

The success of the project will be measured using the following objectives:

Objective 1: Increase the percentage of insurance quotes that convert into active policies by 5–10% within one year by allowing Insurance Agents to focus follow-up efforts on customers who are more likely to decline a quoted policy.

Objective 2: Improve Insurance Agent efficiency by reducing unnecessary follow-ups with customers who are highly likely to accept a quote by 15% within six months.

Objective 3: Support more consistent and informed decision-making by providing predictive insights related to both quote acceptance and quote decline within the existing quoting process.

Project Objectives

The objective of this project is to support better decision-making during the insurance quoting process by providing Insurance Agents with predictive information about whether a customer is likely to accept or decline a quoted insurance policy. This information will help agents prioritize follow-ups and manage their time more effectively.

Existing Software
The current system:
- Generates insurance quotes based on customer-provided information.
- Is used internally by insurance agents during the quoting process.
- Does not provide insights into whether customers are likely to accept or decline a quote.

Requested Enhancement
The requested enhancement will:
- Analyze historical quotes and policy data.
- Predict the likelihood that a customer will accept or decline a quoted insurance policy.
- Display prediction results only to Insurance Agents and internal users.
- Integrate with the existing quoting system without changing customer-facing functionality.

Constraints, Assumptions, Risks, Dependencies

Potential Challenges

The potential challenges in this project are:

Data Challenges
- Historical quote data may be incomplete or inconsistent, which could reduce the accuracy of predictions related to acceptance or decline (Amershi et al., 2019).
- Customer preferences and behaviors may change over time, requiring the prediction model to be updated regularly (Lwakatare et al., 2019).

Technical Challenges
- Integrating a machine learning model that predicts both acceptance and decline into the existing system may require additional testing and validation.
- Monitoring prediction accuracy over time adds complexity to system maintenance (Amershi et al., 2019).

Organizational Challenges
- Insurance Agents may need training to understand and correctly interpret predictions related to customer acceptance and decline.
- Stakeholders may expect exact outcomes, even though machine learning predictions are probabilistic and based on historical patterns.

Risks and Dependencies
- The project depends on access to reliable historical quote and policy data.
- Data privacy, security, and regulatory requirements must be followed.

- Continued collaboration between business and technical teams is required to maintain system effectiveness.

Identifying these challenges early helps reduce project risk and supports effective planning throughout the SDLC (Lwakatare et al., 2019; EBSCO, n.d.).

Project Team

The following team members were integral in the development and contributions to this specification:

Name	Role
Marcela Isabel Redondo Hernandez	Data Scientist

UNIT 2: SOFTWARE DEVELOPMENT METHODOLOGY

The project team for the Quote Acceptance Prediction System (QAPS) at Caribe Insurance Group will use a **hybrid software development methodology** to execute the Software Development Life Cycle (SDLC). This methodology combines elements of a structured development approach with selected Agile practices. A hybrid approach is appropriate because the project includes both stable system components and a machine learning feature that may require ongoing testing and refinement (GeeksforGeeks, n.d.).

The structured portion of the methodology follows a traditional, plan-driven approach similar to Waterfall and is applied during the early phases of the SDLC, particularly during planning and requirements analysis (Atlassian, n.d.). Caribe Insurance Group's existing insurance quoting system has well-defined functionality and business rules that benefit from clear documentation and upfront analysis. Business requirements will be captured in a Business Requirements Document (BRD), including data inputs used in quote generation, user access rules for Insurance Agents, and requirements for displaying prediction results within the internal system. Defining requirements early helps reduce ambiguity, supports coordination between business and technical teams, and minimizes the risk of significant changes later in the project (Reqtest, n.d.; EBSCO, n.d.).

Agile practices will be used during the development and improvement of the machine learning component of QAPS. Machine learning systems depend heavily on data quality and often require experimentation, testing, and adjustment to improve prediction accuracy. Iterative development cycles will allow the project team to test prediction accuracy using historical quote data, review accept and decline predictions with Insurance Agents, and refine the model to improve usefulness over time. This approach supports flexibility and continuous learning, which are important when predicting customer behavior that may change as market conditions or customer preferences evolve (Amershi et al., 2019; Asana, n.d.).

Research supports the use of hybrid methodologies for projects that involve both predictable system elements and data-driven or experimental components. Structured approaches provide stability and control, while Agile practices enable adaptability and responsiveness to change. By combining these approaches, project teams can manage risk while still allowing innovation and improvement throughout the SDLC (Mishra & Alzoubi, 2023).

Several tools and processes will support the hybrid methodology used for QAPS. The BRD will serve as the primary document for managing high-level business requirements. Iterative development cycles will be used for the machine learning feature to allow regular testing and refinement. Stakeholder reviews with Insurance Agents will be conducted to validate prediction usefulness, confirm workflow integration, and ensure follow-up prioritization aligns with business goals. In addition, basic version control practices will be used to track changes to both the software and the prediction model.

In conclusion, a hybrid software development methodology provides the most appropriate approach for the QAPS project. This methodology supports structured planning for stable system components while allowing flexibility and continuous improvement for the machine learning feature. By balancing control and adaptability, the project team increases the likelihood of delivering a system that effectively supports Insurance Agents in making informed follow-up decisions.

UNIT 3: BUSINESS REQUIREMENTS

Functional Requirements List

Req. ID #	Requirement
FR-1	The system must generate an insurance quote based on customer-provided information.
FR-2	The system must predict whether a customer is likely to accept or decline an insurance quote.
FR-3	The system must display prediction results to Insurance Agents within the internal quoting system.
FR-4	The system must restrict prediction results to internal users only.
FR-5	The system must store historical quote and policy outcome data for model training and evaluation.

Non-Functional Requirements List

Usability

NFR-1: The system must allow Insurance Agents to view prediction results without additional training beyond standard onboarding.

Reliability

NFR-2: The system must maintain prediction functionality with no more than 1% failure rate during normal operation.

Performance

NFR-3: Prediction results must be displayed within 3 seconds for at least 90% of all quote requests.

Security & access controls

NFR-4: Only authorized internal users, such as Insurance Agents, must be able to access prediction results.

Supportability

NFR-5: The system must allow Insurance Agents to access quote predictions at any time, including outside normal business hours. System updates and maintenance should be performed with minimal disruption to system availability.

Other

NFR-6: The system must log access to prediction results, including user ID, date, and time, to support auditing and compliance requirements.

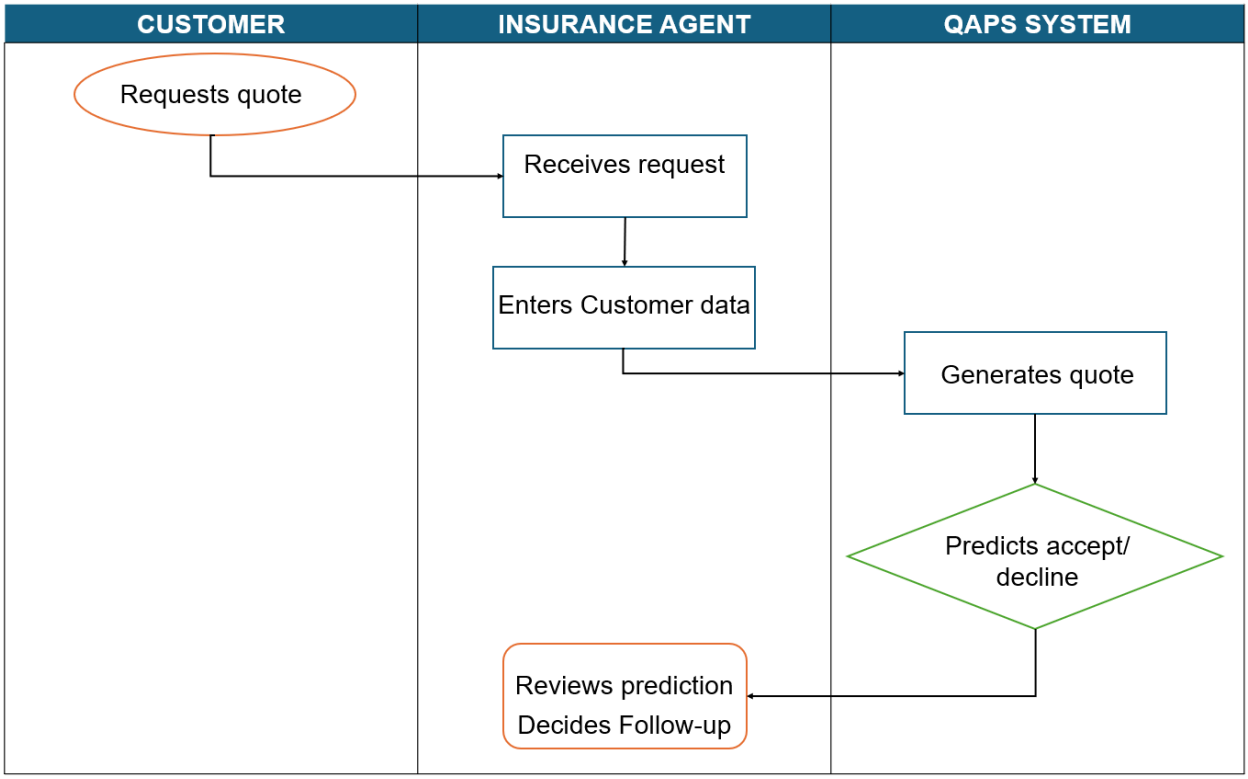
Business Process Diagram

SUMMARY BUSINESS PROCESS NARRATIVE

The business process for the Quote Acceptance Prediction System (QAPS) is modeled using a **swim lane diagram** to illustrate how activities flow across different roles involved in the insurance quoting process. Swim lane diagrams are commonly used in business process modeling because they clearly separate responsibilities while showing the sequence of actions between users and systems (IBM, n.d.; ConceptDraw, n.d.).

The process begins when a customer requests an insurance quote. The Insurance Agent receives the request and enters customer information into the existing quoting system. Once the quote is generated, the QAPS system analyzes relevant quote and historical policy data to predict whether the customer is likely to accept or decline the policy. The prediction result is displayed only to the Insurance Agent and is not visible to the customer. Based on this information, the agent determines the appropriate follow-up action.

Modeling the process in this way helps clarify system interactions, supports communication between stakeholders, and ensures a shared understanding of how QAPS integrates with existing business workflows (IBM, n.d.).



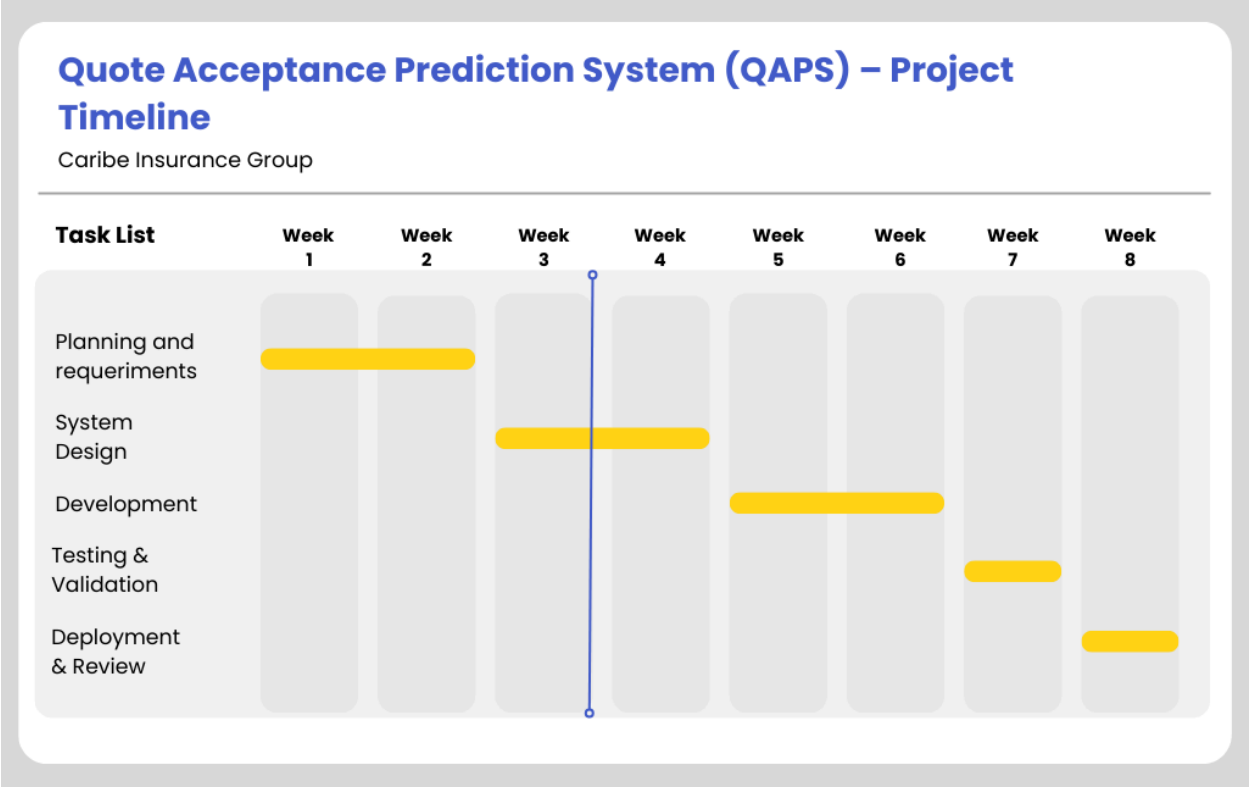
Project Management Timeline

SUMMARY PROJECT MANAGEMENT TIMELINE

A **Gantt-style project timeline** is used to represent the major phases required to build and deploy the Quote Acceptance Prediction System (QAPS). Gantt charts are commonly used in project management because they visually display task sequencing, duration, and dependencies in a clear and easy-to-understand format (Wrike, n.d.).

The project begins with a Planning and Requirements phase, during which business objectives are defined and system requirements are documented in the Business Requirements Document (BRD). This is followed by a Design phase to determine how the prediction system will integrate with the existing insurance quoting system. The Development phase includes building the machine learning model and implementing system enhancements. During the Testing and Validation phase, the system is evaluated for functionality, accuracy, and usability with feedback from Insurance Agents. The system is then released during the Deployment phase, followed by a Review and Support phase to ensure system stability and user adoption.

Using a Gantt-style timeline helps ensure that project activities are completed in the correct order and supports effective planning and coordination among stakeholders (Plane, n.d.; Wrike, n.d.).



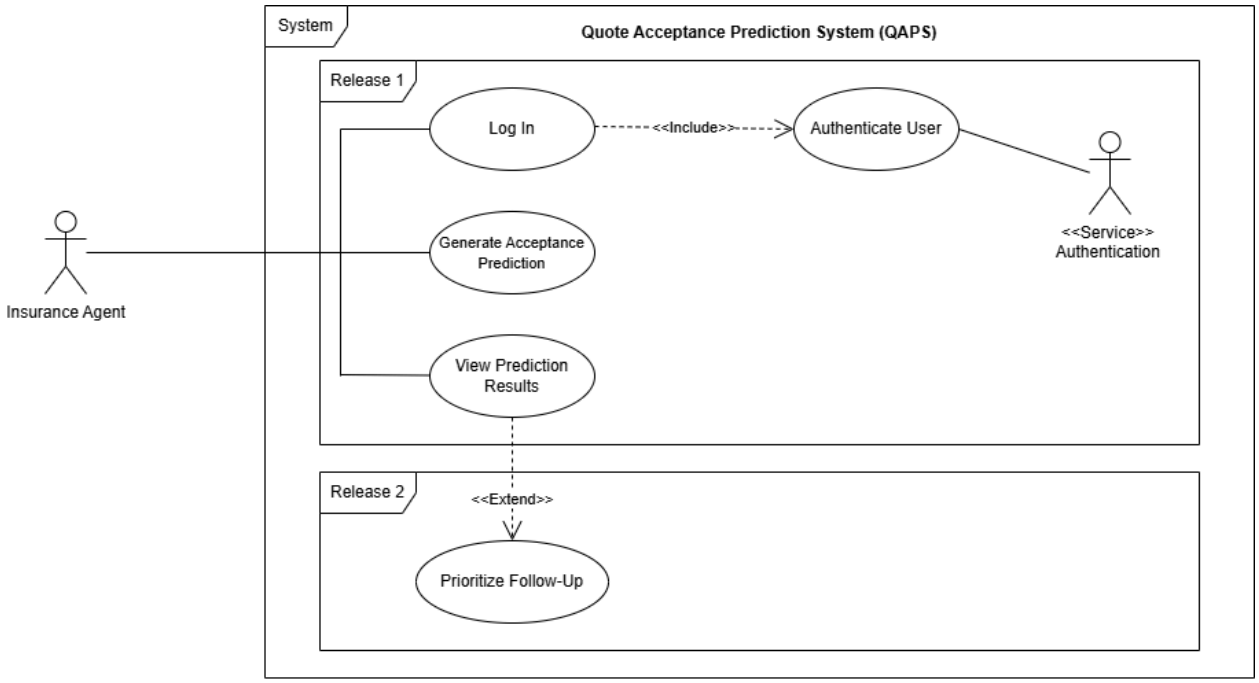
Each phase is aligned to the academic project timeline and reflects the hybrid development approach described in Unit 2, with structured planning early and iterative development and testing later.

UNIT 4: USE CASE DIAGRAM

SUMMARY USE CASE NARRATIVE

This use case diagram illustrates how internal users interact with the Quote Acceptance Prediction System (QAPS), the machine learning technology developed for Caribe Insurance Group. The primary user of the system is the Insurance Agent, who must first log in and authenticate using a valid system identifier before accessing QAPS functionality. Once authenticated, the Insurance Agent can generate and view predictions that indicate whether a customer is likely to accept or decline an insurance quote. These predictions support Insurance Agents in prioritizing customer follow-up activities and improving operational efficiency.

The use case diagram focuses only on internal interactions and does not include the customer as a user of QAPS. This reflects the business requirement that prediction results are visible only to authenticated internal users. Use case diagrams are commonly used in systems analysis to define system scope and clarify user responsibilities at a high level (Visual Paradigm, n.d.).

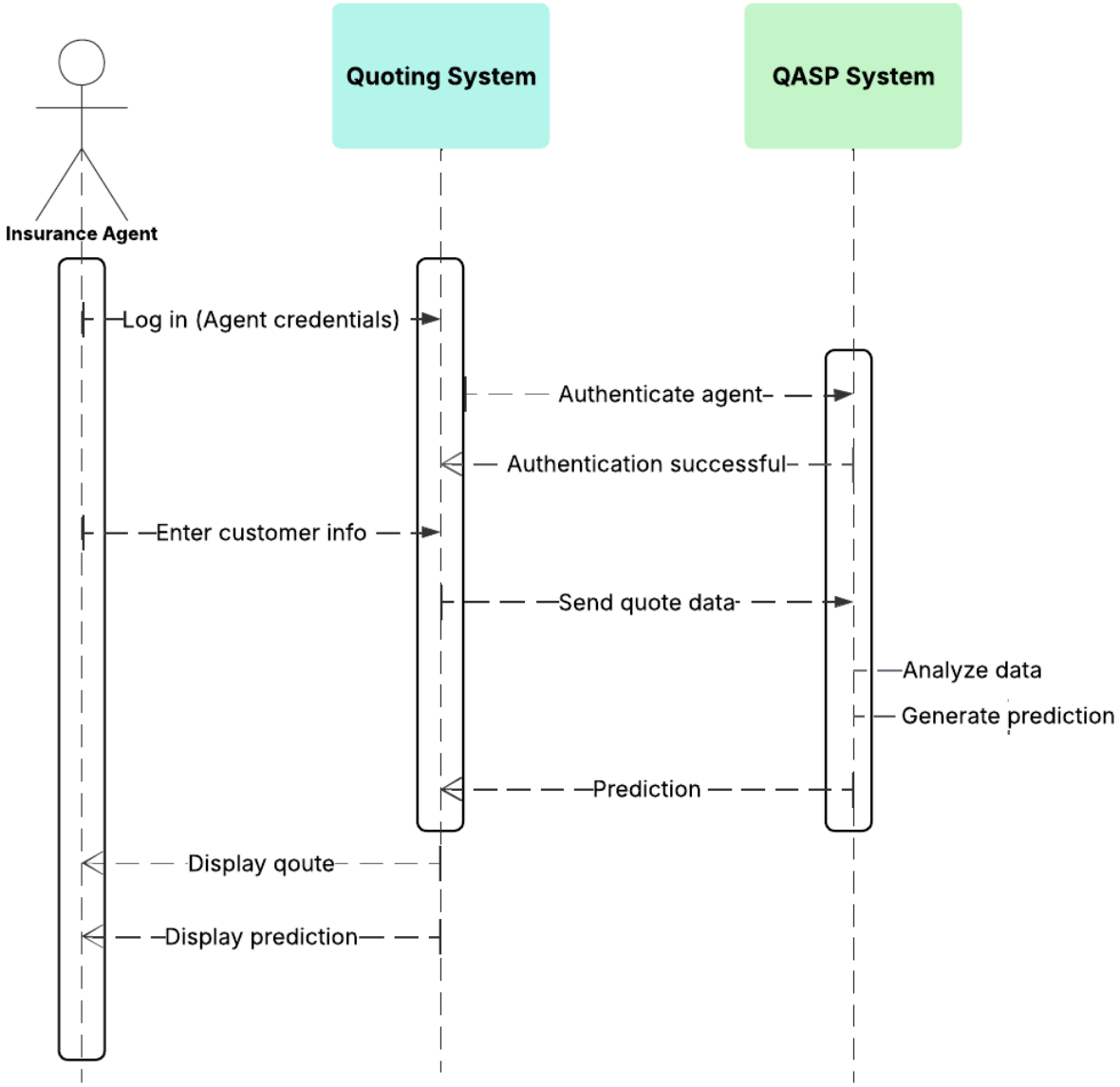


UNIT 4: SEQUENCE DIAGRAM

SUMMARY SEQUENCE DIAGRAM NARRATIVE

This sequence diagram illustrates the order of interactions between the Insurance Agent, the existing insurance quoting system, and the Quote Acceptance Prediction System (QAPS) during a single insurance quote request. The diagram begins with the Insurance Agent logging in and being authenticated using a valid agent identifier. Once authentication is successful, the Insurance Agent enters customer information, which is processed by the quoting system and sent to QAPS to generate a prediction. The prediction result is returned and displayed to support agent decision-making.

The sequence diagram uses UML notation, where stick figures represent actors, vertical lifelines represent system components, and horizontal arrows represent messages exchanged over time. The diagram represents a single, defined quote request scenario and does not include looping or alternate flows, which aligns with the project's hybrid development approach (GeeksforGeeks, nd; Visual Paradigm, nd).



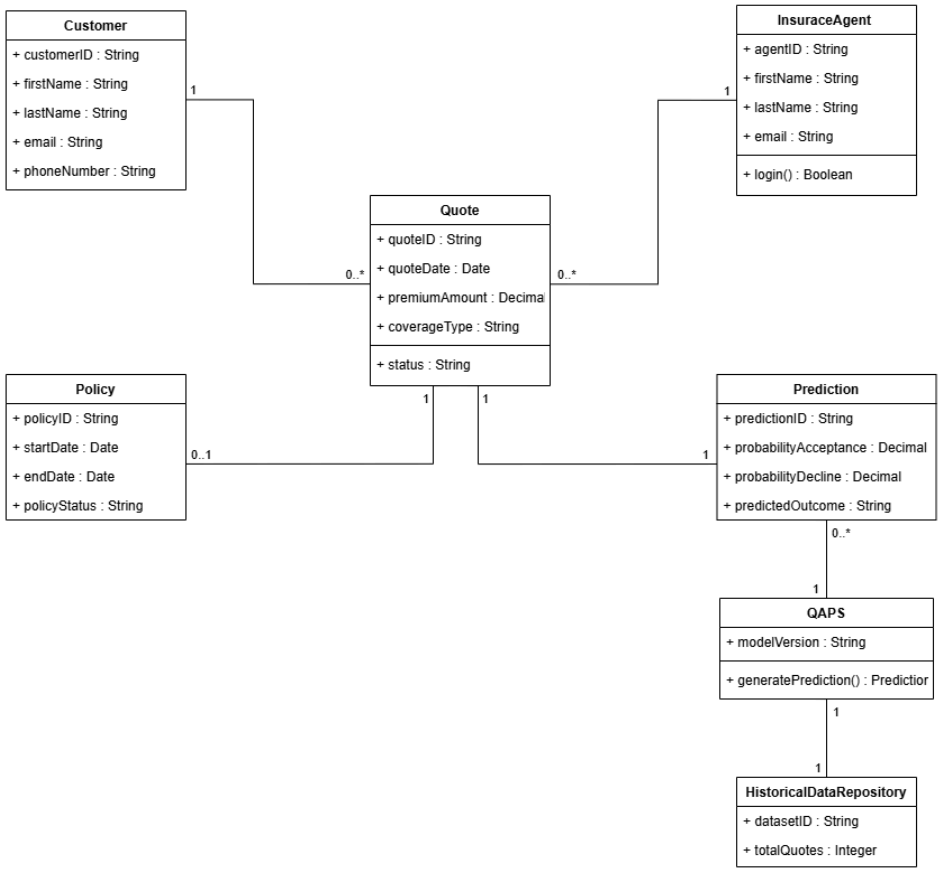
UNIT 5: CLASS DIAGRAM

SUMMARY CLASS DIAGRAM NARRATIVE

This class diagram represents the static structure of the Quote Acceptance Prediction System (QAPS). The diagram models the primary system classes, their attributes with data types, and the relationships between them. Class diagrams are commonly used in systems analysis to provide a structural view of a system and to illustrate how system components are organized before implementation (GeeksforGeeks, n.d.; Visual Paradigm, n.d.).

The diagram includes key business and system entities such as Customer, InsuranceAgent, Quote, Policy, Prediction, QAPS, and HistoricalDataRepository. The Quote class serves as the central entity and connects customers, insurance agents, and prediction results. Each Quote generates one Prediction, and a Quote may optionally result in a Policy. The QAPS class represents the machine learning system responsible for generating predictions using historical data.

Multiplicity values were assigned based on the business rules defined in the Business Requirements Document. A Customer may request multiple Quotes (1 to 0..*), while each Quote is associated with exactly one Customer. Similarly, one InsuranceAgent may generate multiple Quotes (1 to 0..*). Each Quote produces exactly one Prediction (1 to 1), and a Quote may optionally result in one Policy (1 to 0..1). The QAPS system generates multiple Predictions (1 to 0..*) and relies on a single HistoricalDataRepository (1 to 1) for model training and evaluation. Modeling these classes and their associations helps clarify how data is structured within the machine learning enhancement and how it integrates with the existing insurance quoting system.



UNIT 6: DATA DICTIONARY

SUMMARY DATA DICTIONARY NARRATIVE

This data dictionary defines the data elements used within the Quote Acceptance Prediction System (QAPS) and provides a structured description of the information required to support the machine learning technology. The data dictionary documents entities, attributes, data types, field lengths, and data constraints such as primary keys, null values, and uniqueness rules. A data dictionary serves as a structured reference for data definitions that helps ensure consistency, reduce data redundancy, and support clear communication between business and technical users (GeeksforGeeks, n.d.).

The data dictionary was developed based on the classes and attributes identified in the class diagram and includes entities such as Customer, InsuranceAgent, Quote, Policy, Prediction, QAPS, and HistoricalDataRepository. Defining these data elements helps clarify how information is stored and structured within the system and supports accurate systems analysis and design.

The data dictionary also supports the Data Flow Diagram (DFD) by defining the data that moves between external entities, processes, and data stores. Data flow diagrams provide a visual representation of how data enters, is transformed, stored, and exits a system, while the data dictionary provides detailed descriptions of that data (Visual Paradigm, n.d.; IBM, n.d.). Together, these artifacts help improve understanding of system requirements and support alignment between business and technical design.

LEGEND

- **Entity Name** – Logical grouping of related data elements within the system.
- **Entity Description** – Brief explanation of the purpose of the entity.
- **Column Name** – Name of the individual data element (attribute).
- **Column Description** – Definition or purpose of the data element.
- **Data Type** – Type of data stored (eg, string, integer, decimal, date).
- **Length** – Maximum size or precision of the data element when applicable.
- **Primary Key** – Indicates whether the attribute uniquely identifies a record.
- **Nullable** – Indicates whether the field can contain no value.
- **Unique** – Indicates whether duplicate values are allowed within the entity.

Entity Name	Entity Description	Column Name	Column Description	Data Type	Length	Primary Key	Nullable	Unique
Customer	Person requesting an insurance quote	customerID	Unique identifier for customer	string	10	true	false	true
		firstName	Customer first name	string	50	false	false	false
		lastName	Customer last name	string	50	false	false	false
		email	Customer email address	string	100	false	false	false
		phoneNumber	Customer contact number	string	20	false	false	false
Insurance Agent	Internal user who generates quotes and reviews predictions	agentID	Unique identifier for insurance agent	string	10	true	false	true
		firstName	Agent first name	string	50	false	false	false
		lastName	Agent last name	string	50	false	false	false
		email	Agent email address	string	100	false	false	true
Quote	Insurance quote generated for a customer	quoteID	Unique quote identifier	string	10	true	false	true
		quoteDate	Date quote was created	date	date	false	false	false
		premiumAmount	Estimated premium amount	decimal	10,2	false	false	false
		coverageType	Type of insurance coverage	string	50	false	false	false
		status	Quote status (pending/accepted/declined)	string	20	false	false	false

Policy	Insurance policy created after quote acceptance	policyID	Unique policy identifier	string	10	true	false	true
		startDate	Policy start date	date	—	false	false	false
		endDate	Policy end date	date	—	false	false	false
		policyStatus	Current policy status	string	20	false	false	false
Prediction	Machine learning prediction result for a quote	predictionID	Unique prediction identifier	string	10	true	false	true
		probabilityAcceptance	Probability that customer accepts quote	decimal	5,2	false	false	false
		probabilityDecline	Probability that customer declines quote	decimal	5,2	false	false	false
		predictedOutcome	Final predicted result	string	20	false	false	false
Historical Data Repository	Stores historical quote and policy data for model training	datasetID	Unique dataset identifier	integer	10	true	false	true
		totalQuotes	Total historical quotes stored	integer	10	false	false	false
QAPS	Machine learning system that generates predictions	modelVersion	Current machine learning model version	String	20	false	false	false

UNIT 6: DATA FLOW DIAGRAM

SUMMARY DATA FLOW DIAGRAM NARRATIVE

This Data Flow Diagram (DFD) illustrates how data moves through the Quote Acceptance Prediction System (QAPS) using Gane-Sarson notation. Data flow diagrams are used in systems analysis to represent how information enters, is processed, stored, and transmitted between system components and external entities (Visual Paradigm, n.d.; IBM, n.d.). The diagrams focus on the movement of data rather than system behavior or timing, providing a high-level view of how the machine learning enhancement integrates with the insurance quoting workflow.

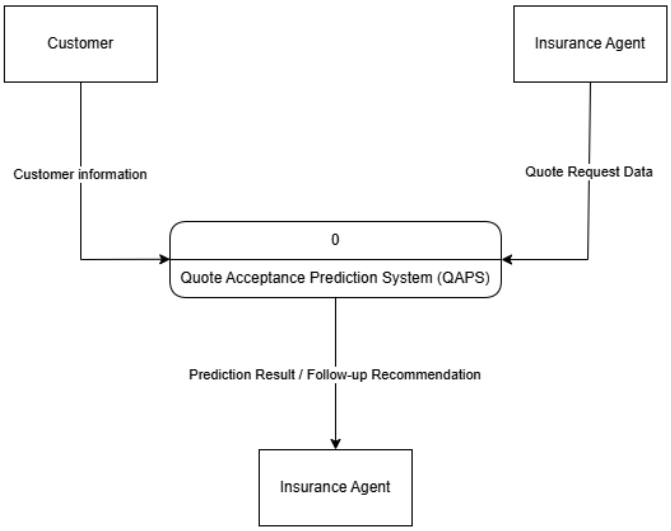
The DFD is presented in three levels of detail to show progressively deeper views of the system. The Context Diagram provides a high-level representation of the overall system by showing external entities and their interactions with QAPS. Diagram 0 expands this view by identifying the major system processes, data stores, and data flows involved in generating insurance quotes, producing predictions, and supporting follow-up actions. Diagram 1 further decomposes the prediction process to illustrate the internal steps required to prepare input data, run the prediction model, and store prediction results.

Across these levels, the diagrams show how customer information is collected by an Insurance Agent, processed through the quoting workflow, and used by QAPS to generate prediction results supported by historical data. Using Gane-Sarson notation helps clearly separate external entities, processes, data stores, and data flows, making the system easier to understand for both business and technical stakeholders (Visual Paradigm, n.d.). These diagrams complement the data dictionary by showing how defined data elements move through processes and data stores within the system.

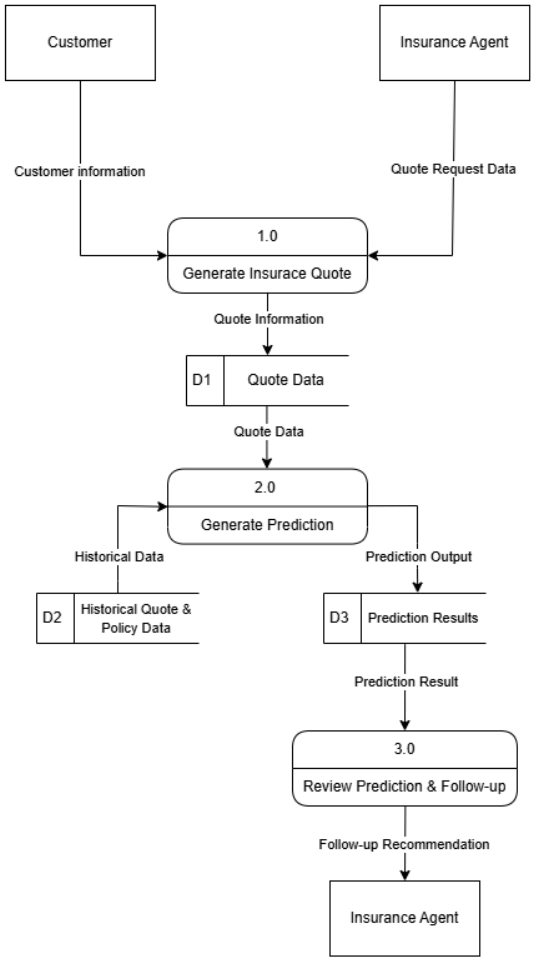
LEGEND

- **External Entity (Rectangle)** – Represents a person or system outside the process boundary that sends or receives data.
- **Process (Gane-Sarson Process Box)** – Represents an activity that transforms input data into output data. Processes are numbered to show logical flow and levels of decomposition.
- **Data Store (Open-Ended Rectangle)** – Represents a location where data is stored for later use.
- **Data Flow (Arrow)** – Represents the movement of data between entities, processes, and data stores.

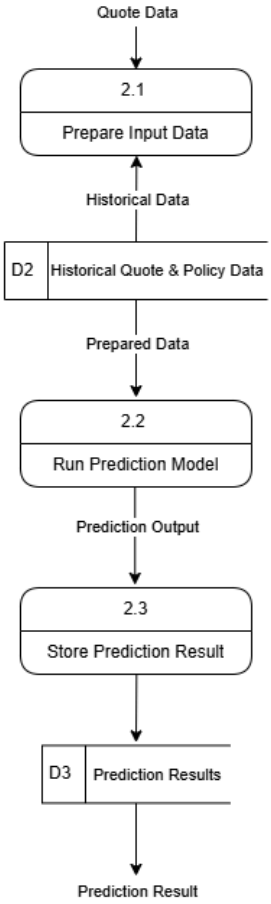
Context Diagram (Level 0 DFD)



Data Flow Diagram — Diagram 0 (Level-0 DFD)



Data Flow Diagram — Diagram 1 (Level-1 DFD: Decomposition of Process 2.0 Generate Prediction)



UNIT 6: ENTITY RELATIONSHIP DIAGRAM (ERD)

SUMMARY ENTITY RELATIONSHIP DIAGRAM (ERD) NARRATIVE

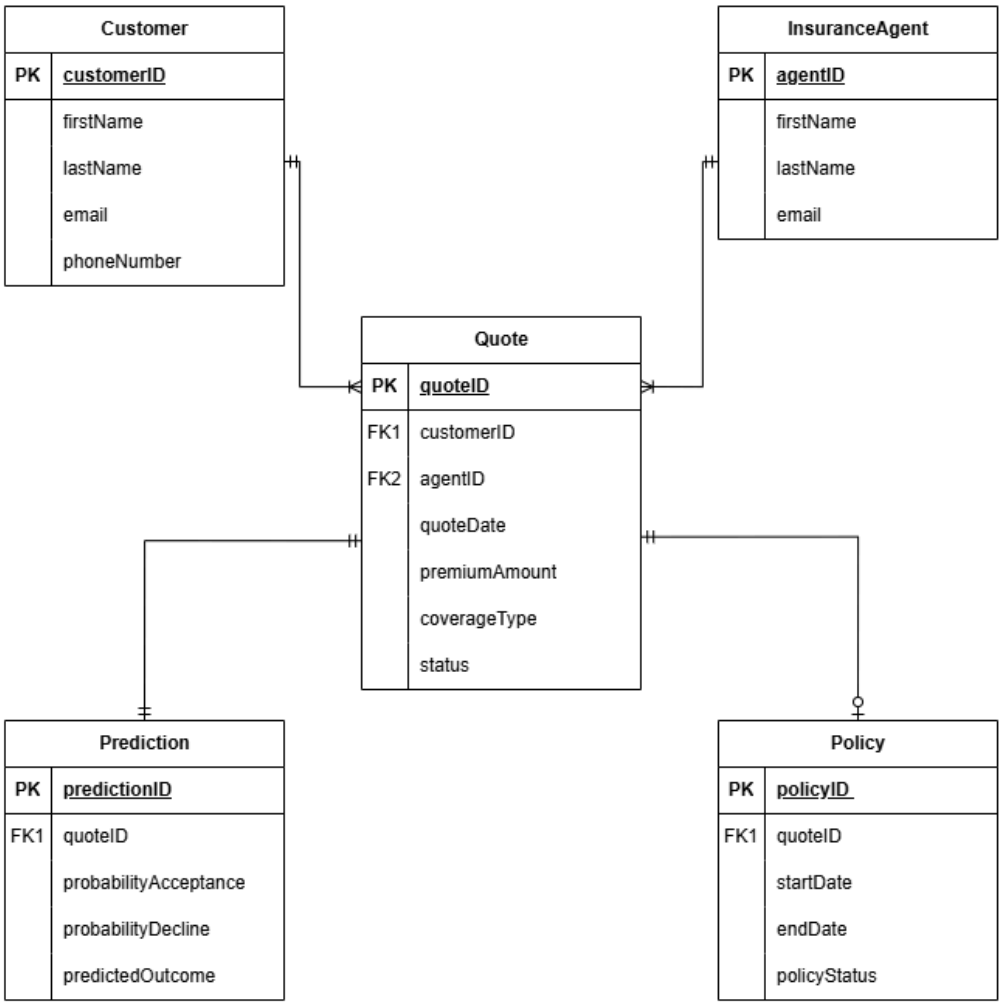
This Entity Relationship Diagram (ERD) illustrates the data structure for the Quote Acceptance Prediction System (QAPS) and shows how core business entities relate within the database design. Entity relationship diagrams are used in systems analysis to represent entities, their attributes, and the relationships between them in order to support database planning and data organization (Atlassian, n.d.). The diagram provides a conceptual and logical view of how data is stored and connected, helping ensure consistency and clarity before implementation.

The ERD includes key entities such as Customer, InsuranceAgent, Quote, Prediction, and Policy. Each entity contains attributes that describe the data stored within the system, including primary keys that uniquely identify records and foreign keys that establish relationships between entities. The Quote entity serves as the central data component, linking customers and insurance agents to predict results and optional policy creation. Relationship cardinality is used to show how entities interact, such as one-to-many relationships between customers and quotes, and one-to-one or optional relationships between quotes, predictions, and policies.

By modeling entities, attributes, and relationships, the ERD supports database design and helps align technical implementation with business requirements. The diagram complements the data dictionary and data flow diagrams by providing a structured representation of how data is organized and maintained within the system (Atlassian, n.d.).

LEGEND

- **Entity (Rectangle/Table)** – Represents a real-world object or concept about which data is stored, such as Customer or Quote.
- **Attribute (Field)** – Represents specific properties or details of an entity, such as names, dates, or status values.
- **Primary Key (PK)** – Attribute that uniquely identifies each record in an entity.
- **Foreign Key (FK)** – Attribute used to establish relationships between entities by referencing a primary key in another entity.
- **Relationship Line (Crow's Foot Notation)** – Represents how entities are connected and shows relationship cardinality (one-to-one, one-to-many, or optional relationships).
- **Crow's Foot Symbol** – Indicates “many” records in a relationship.
- **Circle (O)** – Indicates an optional relationship (zero occurrences allowed).
- **Single Line (|)** – Indicates one mandatory occurrence.


Customer
PK customerID
firstName
lastName
email
phoneNumber
InsuranceAgent
PK agentID
firstName
lastName
email
Quote
PK quoteID
FK1 customerID
FK2 agentID
quoteDate
premiumAmount
coverageType
status
Prediction
PK predictionID
FK1 quoteID
probabilityAcceptance
probabilityDecline
predictedOutcome
Policy
PK policyID
FK1 quoteID
startDate
endDate
policyStatus

UNIT 7: SYSTEM EVALUATION

SYSTEM EVALUATION CRITERIA

Vulnerability

General Description

A vulnerability is a weakness in system design, implementation, or operational controls that can be exploited by a threat actor. Vulnerabilities may exist in software code, authentication mechanisms, system architecture, or data handling processes. If these weaknesses are not identified and addressed, they can result in unauthorized access, data breaches, or system disruption. Vulnerabilities often originate during the Software Development Life Cycle (SDLC), especially when security is not incorporated early in design and development decisions (Moghbel & Modiri, 2011). Wakchaure and Joshi (2011) emphasize that vulnerability identification and analysis should be integrated into each SDLC phase to reduce long-term system risk.

Assessment in the Context of Machine Learning Technology

In the context of QAPS, vulnerabilities may arise from insecure integration of the machine learning model, insufficient input validation, weak authentication mechanisms, or inadequate protection of historical training data. Because QAPS relies on customer data to generate predictive outputs, it may also be exposed to data manipulation or poisoning if safeguards are not properly implemented. OWASP (2021) identifies injection attacks and broken authentication as common weaknesses in modern applications, which could become especially relevant if QAPS evolves into a web-based system.

Mitigation Strategies

To mitigate these risks, the organization should integrate security practices into every SDLC phase. This includes implementing secure coding standards, conducting static and dynamic code analysis, performing penetration testing, and continuously monitoring system performance. Addressing vulnerabilities early in development significantly reduces the likelihood of exploitation after deployment.

Scalability

General Description

Scalability refers to a system’s ability to handle increasing workloads without sacrificing performance or reliability (GeeksforGeeks, n.d.-b). As organizations grow, systems must support more users, higher transaction volumes, and larger datasets efficiently. Scalability can involve expanding processing power, increasing storage capacity, or distributing workloads across multiple servers. Without proper scalability planning, systems may experience slow performance, downtime, or operational inefficiencies.

Assessment in the Context of Machine Learning Technology

In the context of QAPS, scalability is important because the system may eventually support more insurance agents, increased prediction requests, and larger datasets used for model training. Machine learning systems also require computational resources for

retraining models. As historical customer data grows, retraining processes may demand additional processing power and storage capacity. If scalability is not properly addressed, prediction response times may slow down and model updates may become inefficient.

Mitigation Strategies

To address scalability concerns, the organization should consider implementing horizontal scaling by adding additional servers or vertical scaling by increasing hardware capacity (GeeksforGeeks, n.d.-b). Cloud-based infrastructure, load balancing mechanisms, database optimization, and performance monitoring tools can further ensure that QAPS remains responsive and efficient as demand increases.

Maintainability

General Description

Maintainability refers to how easily a system can be modified to correct defects, improve performance, or adapt to new requirements (GeeksforGeeks, n.d.-a). Over time, systems require updates due to evolving business needs, technological changes, and discovered errors. High maintainability reduces long-term operational costs and improves system longevity. Well-designed systems include clear documentation, modular architecture, and testing frameworks to support future changes.

Assessment in the Context of Machine Learning Technology

In the context of QAPS, maintainability is especially important because machine learning models depend on changing data patterns. Customer behavior and market conditions may evolve, which can reduce model accuracy over time. Amershi et al. (2019) explain that machine learning systems require ongoing monitoring, retraining, and version control to maintain effectiveness. Without structured maintenance processes, QAPS may experience performance degradation due to data drift.

Mitigation Strategies

To mitigate maintainability risks, QAPS should incorporate structured maintenance practices. These include maintaining thorough documentation, implementing automated testing, using version control for model updates, scheduling regular retraining cycles, and continuously monitoring model performance. Planning for corrective, adaptive, perfective, and preventive maintenance (GeeksforGeeks, n.d.-a) supports long-term system reliability.

Security

General Description

Security involves protecting systems and data from unauthorized access, disclosure, alteration, or destruction. Modern software applications face risks such as broken access control and insecure authentication (OWASP, 2021). Effective security practices safeguard confidentiality, maintain data integrity, and ensure system availability.

Systems that process personal or financial information require strong security controls to prevent breaches and reputational harm.

Assessment in the Context of Machine Learning Technology

In the context of QAPS, security is critical because the system processes sensitive customer information and generates prediction results that influence agent decision-making. Unauthorized access to predictive outputs or historical training data could result in data breaches or misuse of information. SentinelOne (n.d.-a) explains that role-based access control (RBAC) ensures users only access information necessary for their assigned roles.

Mitigation Strategies

To mitigate security risks, QAPS should implement strong authentication mechanisms, enforce role-based access control, encrypt data in transit and at rest, validate user input, and maintain audit logs to monitor system activity. Regular security testing further strengthens confidentiality and integrity protections.

Legal Issues

General Description

Legal issues in software systems arise when systems fail to comply with regulatory requirements or cause harm due to negligence. Organizations that process personal data must comply with privacy laws and data protection standards. Failure to secure data or prevent misuse may result in liability or legal penalties (DevOps.com, n.d.). Legal accountability also includes preventing discriminatory outcomes and ensuring responsible system governance.

Assessment in the Context of Machine Learning Technology

In the context of QAPS, legal concerns include protection of personally identifiable information (PII), compliance with data privacy regulations, documentation of predictive decision processes, and prevention of discriminatory outcomes. Because QAPS produces probability-based predictions, improper interpretation or misuse of those outputs could expose the organization to regulatory scrutiny or reputational damage.

Mitigation Strategies

To mitigate legal risks, Caribe Insurance Group should maintain detailed audit logs, document model development and updates, implement strong data protection policies, and conduct periodic compliance reviews. These governance measures demonstrate responsible system management and reduce potential liability exposure.

Ethical Concerns

General Description

Ethical concerns in software development involve fairness, transparency, accountability, and responsible system use (TechTarget, n.d.). Ethical risks arise when systems unintentionally disadvantage certain groups or when decision-making processes lack

clarity. In machine learning systems, ethical concerns often relate to biased data, opaque algorithms, and misuse of automated outputs. Addressing ethical considerations supports public trust and responsible innovation.

Assessment in the Context of Machine Learning Technology

In the context of QAPS, ethical risks may arise from biased training data, overreliance on automated predictions, and misinterpretation of probability scores. SentinelOne (n.d.-b) notes that AI systems can be vulnerable to unintended bias and adversarial manipulation. If biased data influences predictions, customers may be treated unfairly.

Mitigation Strategies

To mitigate these ethical concerns, the organization should conduct regular bias testing, ensure human oversight in final decision-making, clearly communicate the limitations of predictive outputs, and establish internal ethical review procedures. Responsible AI governance supports fair and accountable system use.

APPENDIX

Appendix 1 – Glossary of Terms and Acronyms

Access Control: A security mechanism that restricts system access to authorized users based on their assigned roles or permissions.

Actor: A user or external entity that interacts with a system. In UML diagrams, actors are commonly represented using stick figures.

Agile Practices: Flexible development techniques that support iterative work, frequent feedback, and continuous improvement.

Algorithmic Bias: Systematic errors in machine learning predictions caused by unbalanced or unrepresentative training data, which may result in unfair outcomes.

Artificial Intelligence (AI): A field of computer science focused on developing systems that can perform tasks that normally require human intelligence, such as learning, reasoning, and decision-making.

Association: A structural relationship between two classes that indicates how objects of those classes are connected within the system.

Attribute: A characteristic or property of a class that represents data stored within the system and is written in the format attributeName : DataType.

Attribute: A specific data element that describes an entity, such as a name, date, or status.

BRD: Business Requirements Document. A formal document that describes the business needs or new project for a solution.

Business Process Model: A visual or textual representation of how activities flow through a system, showing the sequence of tasks and the roles responsible for them.

Business Process Model and Notation (BPMN): A standardized method used to model business processes in a clear and consistent way so they can be easily understood by both business and technical stakeholders.

Cardinality: The rule that defines how many instances of one entity can be associated with instances of another entity in a relationship.

Class: A blueprint or template that defines a set of attributes and operations representing a system entity within a class diagram.

Class Diagram: A UML structural diagram that represents the static structure of a system by modeling its classes, attributes, operations, and relationships.

Crow's Foot Notation: A visual notation used in ERDs to represent relationship types and cardinality between entities.

Customer: A person who requests an insurance quote from Caribe Insurance Group.

Data Dictionary: A structured reference that defines data elements used in a system, including entities, attributes, data types, and data constraints.

Data Drift (Model Drift): A decline in machine learning model accuracy over time due to changes in data patterns or user behavior.

Data Flow: The movement of data between external entities, processes, and data stores within a data flow diagram.

Data Flow Diagram (DFD): A visual representation that shows how data moves through a system, including processes, data stores, and external entities.

Data Store: A location within a system where data is stored for later use or retrieval.

Encryption: The process of converting data into a secure format to prevent unauthorized access during storage or transmission.

Entity: A real-world object or concept about which data is stored within a database, such as a customer or quote.

Entity Relationship Diagram (ERD): A diagram used to represent the structure of data within a system by showing entities, attributes, and relationships between them.

External Entity: A person, organization, or external system that sends data to or receives data from the system.

Foreign Key (FK): An attribute that connects one entity to another by referencing the primary key of a related entity.

Functional Requirement: A requirement that defines what the system must do to support business processes and user needs.

Gane-Sarson Notation: A standardized diagramming notation used to represent processes, data stores, external entities, and data flows within a data flow diagram.

Gantt Chart: A project management tool that displays tasks or phases along a timeline to show start dates, durations, and sequence of activities.

Horizontal Scaling (Scale-Out): Increasing system capacity by adding additional servers or nodes to distribute workload.

Insurance agent: An internal user who generates insurance quotes and uses system information to follow up with customers.

Lifeline: A vertical line in a sequence diagram that represents a participant in the interaction, such as a user or system component, over time.

Machine Learning (ML): A subset of artificial intelligence that enables systems to learn from historical data and generate predictions or classifications without being explicitly programmed for each decision.

Maintainability: The ability of a system to be easily modified to fix errors, improve performance, or adapt to new business requirements.

Message: A communication or data exchange between participants in a sequence diagram, represented by arrows that indicate the direction and order of interaction.

Milestone: A significant point or event in a project timeline that indicates the completion of a major phase or deliverable.

ML: Machine Learning. A type of software that uses past data to help predict future outcomes.

Multiplicity: A UML notation that specifies the number of instances of one class that may be associated with another class, such as 1, 0..1, or 0..*.

Non-Functional Requirement: A requirement that defines how the system performs its functions, including quality attributes such as usability, performance, security, and supportability.

One-to-Many Relationship: A relationship where one instance of an entity can be associated with multiple instances of another entity.

One-to-One Relationship: A relationship where one instance of an entity is associated with exactly one instance of another entity.

Operation: A function or method associated with a class that represents system behavior. Operations may include parameters and a return type.

Optional Relationship: A relationship where participation of an entity instance is not required, meaning zero occurrences are allowed.

Penetration Testing: A controlled security assessment that simulates cyberattacks to identify system weaknesses.

Personally Identifiable Information (PII): Any information that can be used to identify an individual, such as name, address, contact details, or financial data.

Plan-Driven Approach: A development approach that emphasizes upfront planning, documentation, and clearly defined project phases.

Predictive analytics: use of historical data and statistical methods to estimate future outcomes.

Primary Key (PK): A unique identifier used to distinguish each record within an entity.

Process: An activity within a data flow diagram that transforms input data into output data.

Project Management Timeline: A visual or structured representation of project phases, durations, and dependencies used to plan and track progress throughout the project lifecycle (Plane, n.d.; Wrike, n.d.).

QAPS: Quote Acceptance Prediction System. The proposed system that predicts whether a customer is likely to accept or decline an insurance quote.

Quote: An estimated cost provided to a customer for an insurance policy based on submitted information.

Quote acceptance: A situation in which a customer agrees to purchase an insurance policy after receiving a quote.

Quote decline: A situation in which a customer chooses not to purchase an insurance policy after receiving a quote.

Role-Based Access Control (RBAC): A security model that grants system access based on a user's role within an organization.

Scalability: The ability of a system to handle increased demand, users, or data volume without losing performance or reliability.

SDLC: Software Development Life Cycle. A structured process or methodology used to plan, design, develop, test, and deploy software systems.

Secure Software Development Life Cycle (Secure SDLC): An approach to software development that integrates security practices into every phase of system design and implementation.

Sequence Diagram: A UML diagram that represents the order of interactions between users and system components over time, showing how messages are exchanged to complete a process (GeeksforGeeks, n.d.; Visual Paradigm, n.d.).

Stakeholder: Any individual or group with an interest in the project, such as, business leaders, insurance agents, and technical teams.

Supportability: A quality attribute that describes how easily a system can be maintained, updated, and supported over time without disrupting users.

Swim Lane Diagram: A type of process diagram that organizes activities into lanes based on roles or systems, helping clarify responsibilities and interactions within a process (IBM, n.d.; ConceptDraw, n.d.).

System Boundary: A visual element in a use case diagram that defines the scope of a system and separates system functionality from external actors (Visual Paradigm, n.d.).

System enhancement: An improvement made to an existing software system to add new functionality or capabilities.

Unified Modeling Language (UML): A standardized visual modeling language used in systems analysis to represent system interactions and behavior (Visual Paradigm, n.d.).

Use Case Diagram: A UML diagram that shows how users (actors) interact with a system by illustrating system functionality and user responsibilities at a high level (Visual Paradigm, n.d.).

User: An internal employee, such as an insurance agent, who interacts with the system.

Visibility: A UML notation symbol (+, -, #) that indicates the access level of class attributes and operations.

Vulnerability: A weakness in a system's design or implementation that could be exploited to compromise security or system performance.

Waterfall Methodology: A traditional, plan-driven software development approach in which project phases are completed in a defined sequence.

Note: The definitions in this glossary are adapted from standard systems analysis, software engineering, cybersecurity, and machine learning terminology based on the sources referenced throughout this project (Amershi et al., 2019; GeeksforGeeks, n.d.; Moghbel & Modiri, 2011; OWASP, 2021; SentinelOne, n.d.; TechTarget, n.d.; Wakchaure & Joshi, 2011).

Appendix 2 - References

- Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software engineering for machine learning: A case study. Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice. IEEE Press, 2019. https://www.microsoft.com/en-us/research/wp-content/uploads/2019/03/amershi-icse-2019_Software_Engineering_for_Machine_Learning.pdf
- Asana. (n.d.). Agile methodology. https://asana.com/es/resources/agile-methodology
- Atlassian. (n.d.). Waterfall methodology. https://www.atlassian.com/agile/project-management/waterfall-methodology
- Atlassian. (n.d.). Entity relationship diagram (ERD). https://www.atlassian.com/work-management/project-management/entity-relationship-diagram
- Black Duck. (n.d.). Top 4 software development methodologies. https://www.blackduck.com/blog/top-4-software-development-methodologies.html
- ConceptDraw. (n.d.). Swim lane diagram examples. https://www.conceptdraw.com/examples/swim-lane
- DevOps.com. (n.d.). Understanding liability in software development: Minimizing legal risks. https://devops.com/understanding-liability-in-software-development-minimizing-legal-risks/
- EBSCO. (n.d.). Systems analysis and design: Creating systems support. https://www.ebsco.com/research-starters/business-and-management/systems-analysis-and-design-creating-systems-support
- GeeksforGeeks. (n.d.-a). Maintainability in system design. https://www.geeksforgeeks.org/system-design/maintainability-in-system-design/
- GeeksforGeeks. (n.d.). Unified Modeling Language (UML) class diagrams. https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/
- GeeksforGeeks. (n.d.). Unified modeling language (UML) sequence diagrams . https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/
- GeeksforGeeks. (n.d.). What is data dictionary? https://www.geeksforgeeks.org/dbms/what-is-data-dictionary/
- GeeksforGeeks. (n.d.). What is hybrid project management? https://www.geeksforgeeks.org/business-studies/what-is-hybrid-project-management/

- GeeksforGeeks. (n.d.-b). What is scalability? https://www.geeksforgeeks.org/system-design/what-is-scalability/

- IBM. (n.d.). Business process model and notation (BPMN). https://www.ibm.com/think/topics/bpmn

- IBM. (n.d.). What is a data flow diagram (DFD)? https://www.ibm.com/think/topics/data-flow-diagram

- Lwakatare, L. E., Raj, A., Bosch, J., Olsson, H. H., & Crnkovic, I. (2019). A taxonomy of software engineering challenges for machine learning systems: An empirical investigation. In International Conference on Agile Software Development (pp. 227–245). Springer, Cham. https://link.springer.com/chapter/10.1007/978-3-030-19034-7_14

- Mishra, A., & Alzoubi, Y. I. (2023). Structured software development versus agile software development: A comparative analysis. Springer. https://link.springer.com/article/10.1007/s13198-023-01958-5

- Moghbel, Z., & Modiri, N. (2011). A framework for identifying software vulnerabilities within SDLC phases. International Journal of Computer Science and Information Security, 9(6), 203–207. https://www.oalib.com/research/2639506

- OWASP Foundation. (2021). OWASP top ten. https://owasp.org/www-project-top-ten/

- Plane. (n.d.). What is a project timeline? https://plane.so/blog/what-is-project-timeline

- Reqtest. (n.d.). Business requirements document (BRD). https://reqtest.com/en/knowledgebase/business-requirements-document-brd/

- SentinelOne. (n.d.-b). The top AI security risks. https://www.sentinelone.com/es/cybersecurity-101/data-and-ai/ai-security-risks/

- SentinelOne. (n.d.-a). What is access control? https://www.sentinelone.com/es/cybersecurity-101/cybersecurity/what-is-access-control/

- TechTarget. (n.d.). Examples of ethical issues in software development. https://www.techtarget.com/searchsoftwarequality/tip/5-examples-of-ethical-issues-in-software-development

- Visual Paradigm. (n.d.). Data flow diagram (DFD) tutorial. https://www.visual-paradigm.com/tutorials/data-flow-diagram-dfd.jsp

- Visual Paradigm Online. (n.d.). Gane-Sarson DFD tutorial. https://online.visual-paradigm.com/knowledge/software-design/gane-sarson-dfd-tutorial

- Visual Paradigm. (n.d.). What is a class diagram? https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/

- Visual Paradigm. (n.d.). What is a sequence diagram? https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/

- Visual Paradigm. (n.d.). What is a use case diagram? https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-use-case-diagram/

- Wakchaure, M. A., & Joshi, S. D. (2011). A framework to detect and analyze software vulnerabilities: Analysis phase in SDLC. International Journal of Advanced Computer Engineering and Architecture, 1(1), 73–83. https://www.semanticscholar.org/paper/A-Framework-to-Detect-and-Analyze-Software-%3A-Phase-Wakchaure-Joshi/216a98f422147ec7e6bb3342eebb5590d92c8324

- Wrike. (n.d.). How to create a project timeline. https://www.wrike.com/blog/how-to-create-project-timeline/